```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import urllib.request

# ---------------------------------------------------------
# INPUT IMAGE
# ---------------------------------------------------------
url = "https://raw.githubusercontent.com/opencv/opencv/master/samples/data/butterfly.jpg"
urllib.request.urlretrieve(url, "input.jpg")

img = cv2.imread("input.jpg")

if img is None:
    raise Exception("Image could not be loaded")

gray1 = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# ---------------------------------------------------------
# ROTATE IMAGE
# ---------------------------------------------------------
h, w = gray1.shape

center = (w // 2, h // 2)

matrix = cv2.getRotationMatrix2D(
    center,
    30,
    1.0
)

rotated = cv2.warpAffine(
    gray1,
    matrix,
    (w, h)
)

# ---------------------------------------------------------
# ADD GAUSSIAN NOISE
# ---------------------------------------------------------
noise = np.random.normal(
    0,
    25,
    rotated.shape
).astype(np.float32)

noisy = rotated.astype(np.float32) + noise

noisy = np.clip(
    noisy,
    0,
    255
).astype(np.uint8)

# ---------------------------------------------------------
# SIFT
# ---------------------------------------------------------
sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(
    gray1,
    None
)

kp2, des2 = sift.detectAndCompute(
    noisy,
    None
)

print("Original features :", len(kp1))
print("Transformed features :", len(kp2))

# ---------------------------------------------------------
# FLANN MATCHER
# ---------------------------------------------------------
index_params = dict(
    algorithm=1,
    trees=5
)

search_params = dict(
    checks=50
)

flann = cv2.FlannBasedMatcher(
    index_params,
    search_params
)

matches = flann.knnMatch(
    des1,
    des2,
    k=2
)

# ---------------------------------------------------------
# RATIO TEST
# ---------------------------------------------------------
good_matches = []

for m, n in matches:

    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

print("Total matches :", len(matches))
print("Good matches  :", len(good_matches))

# ---------------------------------------------------------
# MATCHING RATIO
# ---------------------------------------------------------
matching_ratio = (
    len(good_matches) /
    max(len(kp1), 1)
) * 100

print(
    f"Matching ratio = {matching_ratio:.2f}%"
)

# ---------------------------------------------------------
# DRAW MATCHES
# ---------------------------------------------------------
match_image = cv2.drawMatches(
    gray1,
    kp1,
    noisy,
```

```
    kp2,
    good_matches[:50],
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(16, 8))

plt.imshow(
    cv2.cvtColor(
        match_image,
        cv2.COLOR_BGR2RGB
    )
)

plt.title(
    f"SIFT Matching: Rotation = 30°, Noise = Gaussian"
)
plt.axis("off")

plt.savefig("Q29_feature_matching.png")
plt.show()
```

```
Original features : 1124
Transformed features : 1650
Total matches : 1124
Good matches  : 354
Matching ratio = 31.49%
```